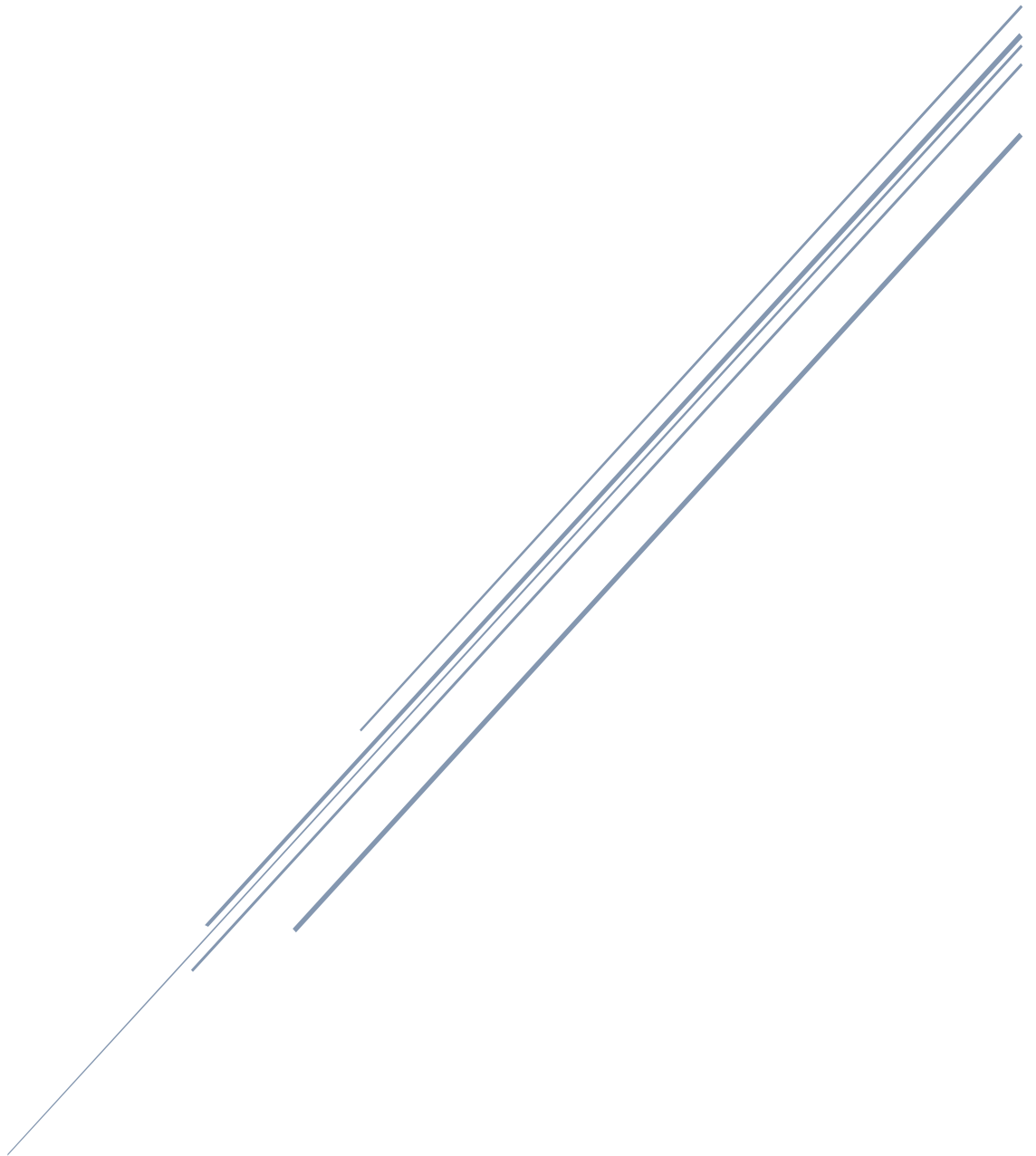


CHAPTER 5

Convolutional Neural Networks



The previous chapter presented the supervised learning pipeline for deep neural networks, focusing on fully connected architectures with a single path from input to output. In contrast, this chapter introduces more specialized components that use sparse connections, shared weights, and parallel processing. It focuses on convolutional layers, which are primarily designed for image data processing.

Images have three key properties that require specialized model architectures:

- **High dimensionality:** A typical image (e.g., 224×224 with RGB channels) has over 150,000 inputs. Fully connected networks would require an extremely large number of parameters, leading to heavy demands in data, memory, and computation.
- **Local correlations:** Nearby pixels are strongly related, but fully connected networks ignore this structure and treat all inputs equally, without considering spatial proximity.
- **Shift invariance:** The meaning of an image remains the same under small shifts, but fully connected models must relearn the same patterns at every position, which is inefficient.

Convolutional layers address these issues by processing local regions independently and sharing parameters across the image. This reduces the number of parameters, captures spatial relationships, and avoids relearning patterns at different positions. Networks built mainly from these layers are called convolutional neural networks (CNNs).

5.1. Invariance and equivariance

Some image properties remain unchanged under certain transformations. We can express this more formally: a function $f(x)$ is invariant to a transformation $t(x)$ if applying the transformation does not change the output.

$$f[t(x)] = f(x)$$

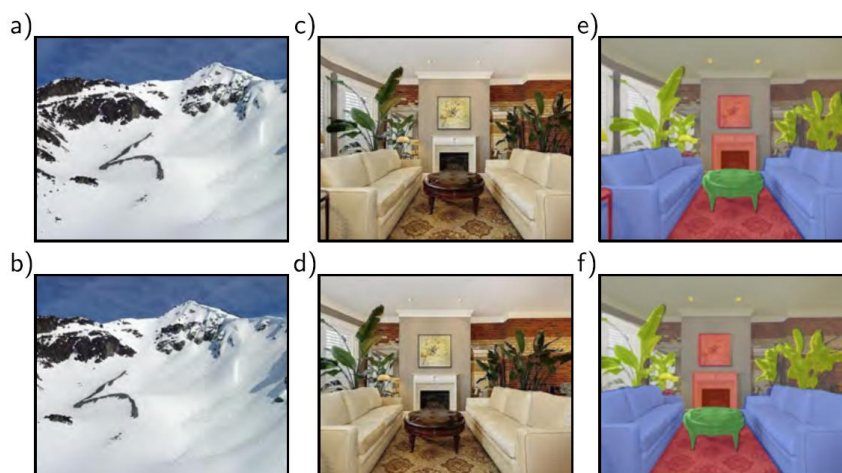
In other words, $f(x)$ produces the same result whether or not the image is transformed. For image classification, models should be invariant to geometric changes. The network should recognize the same object even if the image is shifted, rotated, flipped, or slightly distorted.

A function $f(x)$ is equivariant (or covariant) to a transformation $t(x)$ if its output changes in the same way as the input when the transformation is applied.

$$f[t(x)] = t[f(x)]$$

In other words, applying a transformation to the input results in a corresponding transformation in the output. For tasks like per-pixel image segmentation, networks should be equivariant: if the image is shifted, rotated, or flipped, the resulting segmentation should undergo the same transformation.

CNNs are built from multiple convolutional layers, each preserving translation equivariance. They often also include pooling operations, which introduce a degree of translation invariance.



5.2. Convolutional networks for 1D inputs

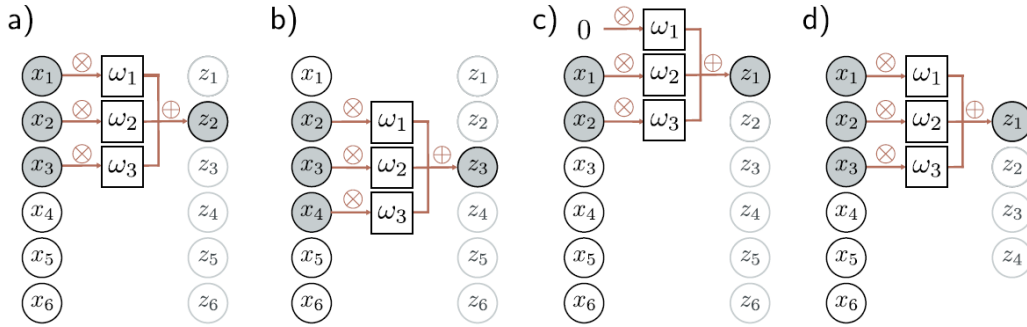
CNNs for 1D signals can be applied to data such as financial time series, audio, or text.

5.2.1. 1D convolution operation

In the 1D case, a convolution operation maps an input vector x to an output vector z , where each element z_i is computed as a weighted sum of neighboring input values. The same set of weights, called the kernel or filter, is applied across all positions. The number of neighboring inputs involved is known as the kernel size. For example, with a kernel size of 3:

$$z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}$$

where $\omega = [\omega_1, \omega_2, \omega_3]^T$ is the convolution kernel. Note that convolution is translation-equivariant: shifting the input x results in an equivalent shift in the output z .



a) Padding

The equation shows that each output value is computed as a weighted sum of the previous, current, and next input elements. This raises an issue at the boundaries: the first position has no previous input, and the last position has no next input.

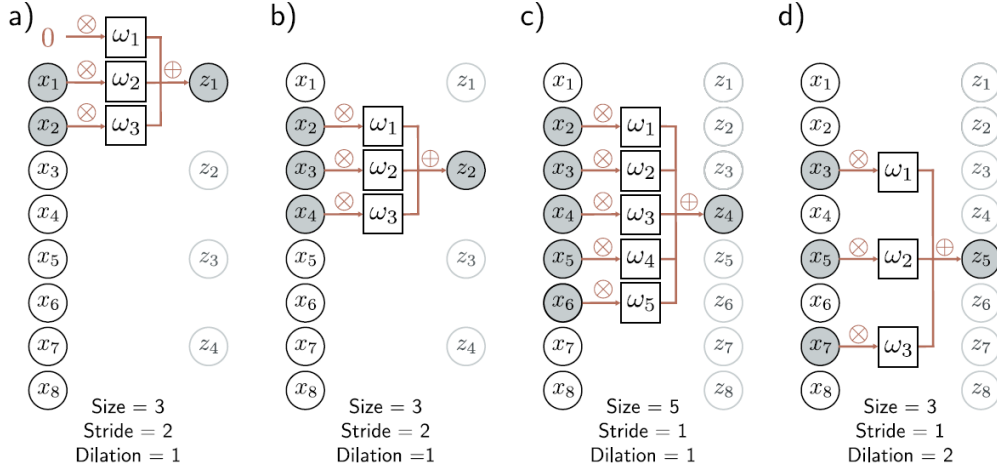
There are two common solutions. The first is padding, where extra values are added at the edges so the computation can proceed normally. In zero-padding, the input is assumed to be zero outside its range. Other options include reflecting the input at the borders.

The second solution is to compute only where the kernel fully overlaps the input, ignoring boundary positions. This is called a valid convolution. It avoids adding artificial information but reduces the size of the output.

b) Stride, kernel size, and dilation

In the previous example, each output was computed from three neighboring inputs, but this is just one case within a broader family of convolution operations. These operations differ mainly in their stride, kernel size, and dilation rate:

- **Stride:** A stride of one means the kernel is applied at every position. With a larger stride (e.g., two), the kernel skips positions, producing fewer outputs.
- **Kernel size:** Increasing the kernel size allows the model to capture information from a wider region. It is usually kept odd so it can be centered on the current position.
- **Dilation:** Larger kernels require more weights, which can be costly. Dilated (or atrous) convolutions address this by inserting gaps (zeros) between kernel elements. This expands the receptive field without increasing the number of parameters. The number of inserted gaps defines the dilation rate.



c) Convolution layer

A convolutional layer produces its output by applying a convolution to the input, adding a bias term β , and then passing the result through an activation function $a(\cdot)$. For a kernel size of three, stride one, and dilation rate one, the i^{th} hidden unit h_i is computed as:

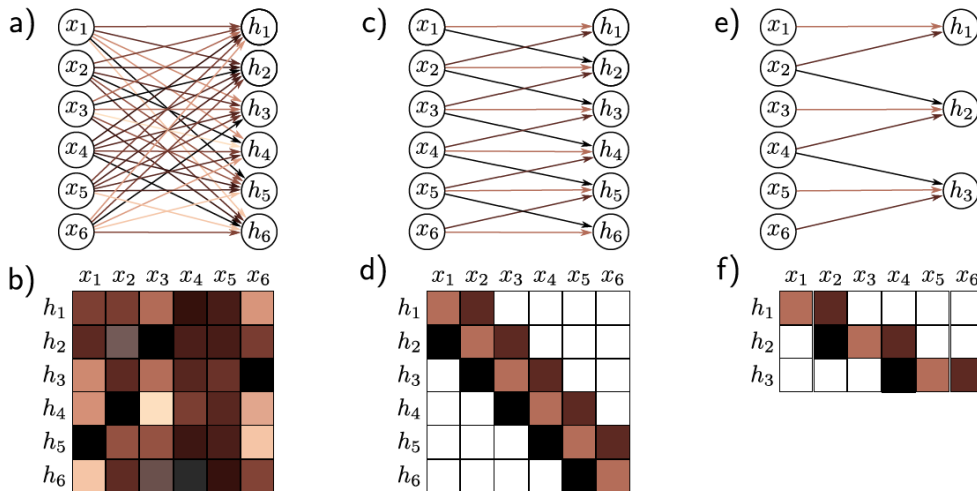
$$h_i = a[\beta + \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}] = a\left[\beta + \sum_{j=1}^3 \omega_j x_{i+j-2}\right]$$

where the bias β_i and kernel weights $\omega_1, \omega_2, \omega_3$ are learnable parameters. With zero-padding, the input x is assumed to be zero outside its valid range.

This formulation can also be viewed as a special case of a fully connected layer, where the i^{th} hidden unit is computed as:

$$h_i = a\left[\beta_i + \sum_{j=1}^D \omega_{ij} x_j\right]$$

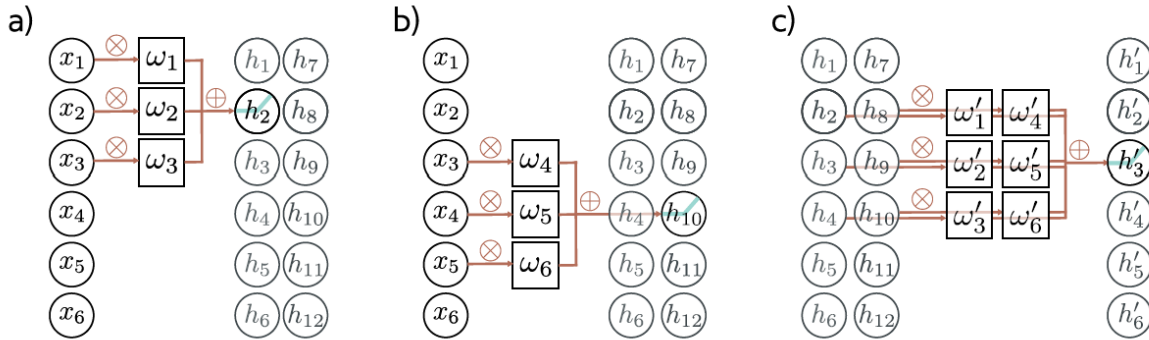
If there are D input units x and D hidden units h , a fully connected layer would require D^2 weights ω and D bias terms β . In contrast, the convolutional layer shares parameters across positions and therefore uses only three weights and a single bias.



d) Channels

Applying only a single convolution can lead to information loss, since the operation aggregates nearby inputs and the ReLU activation suppresses negative values. For this reason, convolutional layers typically compute multiple convolutions in parallel. Each convolution produces its own set of hidden activations, known as a feature map or channel.

The figure below shows an example with two kernels of size three and zero-padding. The first kernel performs a weighted sum over each group of three neighboring pixels, adds a bias, and applies an activation function to produce hidden units h_1 to h_6 , forming the first channel. The second kernel applies a different set of weights and bias to generate another set of activations h_7 to h_{12} , which form the second channel.



In general, both the input and hidden layers consist of multiple channels. If the input layer has C_i channels and we use a kernel of size K for each channel, then each output unit is computed as a weighted sum over all C_i input channels and the K spatial positions, using a weight matrix $\Omega \in \mathbb{R}^{C_i \times K}$ along with a bias term.

If the next layer contains C_o output channels, then the full set of parameters consists of $\Omega \in \mathbb{R}^{C_i \times C_o \times K}$ weights and $\beta \in \mathbb{R}^{C_o}$ biases.

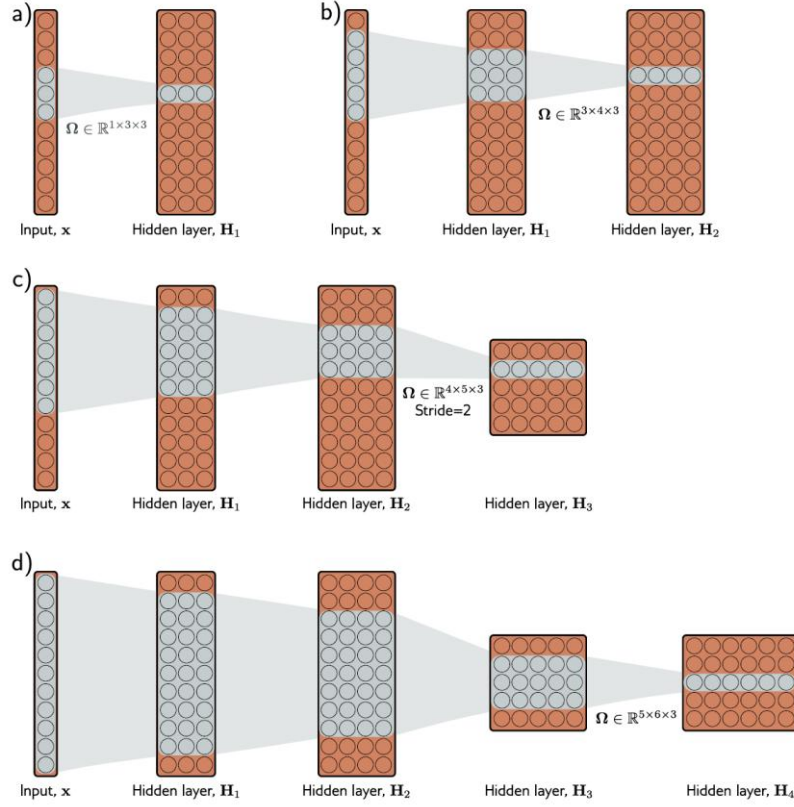
Input: (N, C_{in}, L_{in}) or (C_{in}, L_{in})

Output: (N, C_{out}, L_{out}) or (C_{out}, L_{out}) , where:

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

e) Convolutional networks and receptive fields

The receptive field of a hidden unit refers to the portion of the original input that influences its value. Consider a convolutional network where each layer uses a kernel size of three. In the first layer, each hidden unit depends on three neighboring input values, so its receptive field is size three. In the second layer, each unit combines three adjacent activations from the previous layer, each of which already summarizes three inputs. As a result, the receptive field expands to size five.



5.2.2. Convolutional networks for 2D inputs

In practice, convolutional networks are most commonly used for 2D image data. In this case, the convolutional kernel becomes a two-dimensional filter. A 3×3 kernel $\Omega \in \mathbb{R}^{3 \times 3}$ applied to a 2D input x_{ij} produces hidden activations h_{ij} as follows:

Each output value is computed as a weighted sum over a local 3×3 neighborhood of the input, where the weights are given by the kernel entries ω_{mn} .

The kernel is then slid across the input image both horizontally and vertically, producing an output value at every spatial location.

$$h_{ij} = a \left[\beta + \sum_{m=1}^3 \sum_{n=1}^3 \omega_{mn} x_{i+m-2, j+n-2} \right]$$

In many cases, the input is an RGB image, which can be viewed as a 2D signal with three channels. A 3×3 convolutional kernel in this setting has $3 \times 3 \times 3$ weights, since it operates across all three input channels at each spatial location. Applying this kernel (with zero-padding) produces a 2D output feature map with the same height and width as the input image.

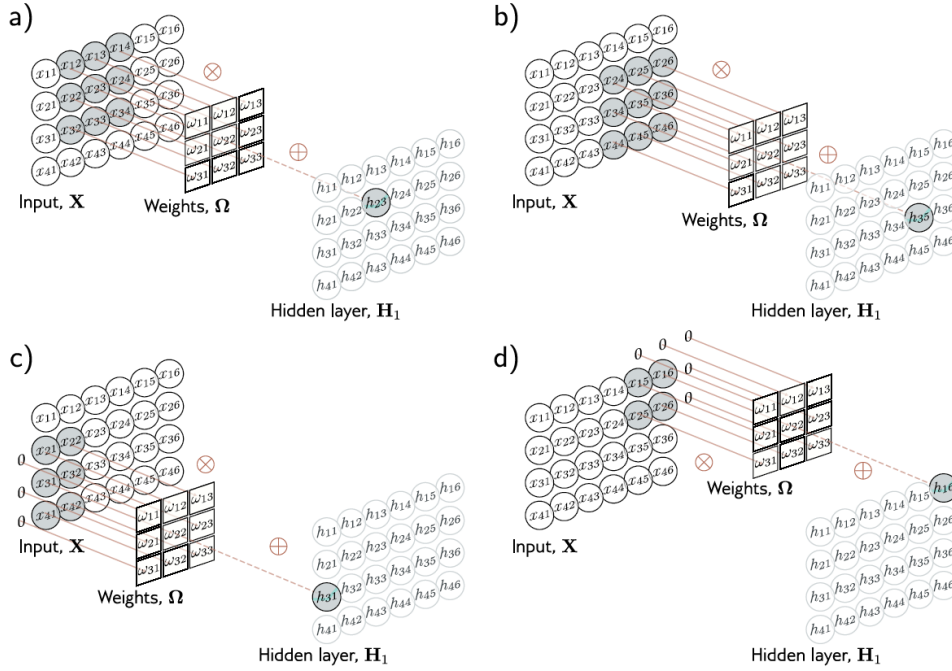
To obtain multiple output channels, we use several different kernels in parallel and stack their outputs along a new dimension, forming a 3D tensor. More generally, for a kernel of size $K \times K$ and C_i input channels, each output value is computed as a weighted sum over $C_i \times K \times K$ inputs plus a bias term. Therefore, producing C_o output channels requires $C_i \times C_o \times K \times K$ weights and C_o bias terms.

Input: $(N, C_{in}, H_{in}, W_{in})$ or (C_{in}, H_{in}, W_{in})

Output: $(N, C_{out}, H_{out}, W_{out})$ or $(C_{out}, H_{out}, W_{out})$, where:

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times padding[0] - dilation[0] \times (kernel_size[0] - 1) - 1}{stride[0]} + 1 \right\rfloor$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times padding[1] - dilation[1] \times (kernel_size[1] - 1) - 1}{stride[1]} + 1 \right\rfloor$$

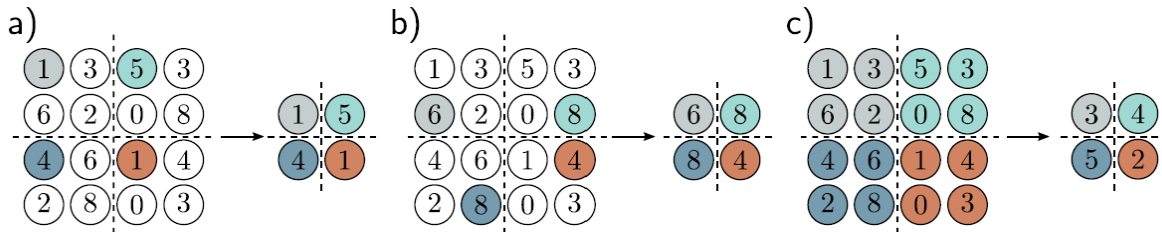


a) Downsampling

There are three common ways to reduce the spatial size of a 2D representation. We focus on the typical case where both height and width are reduced by a factor of two.

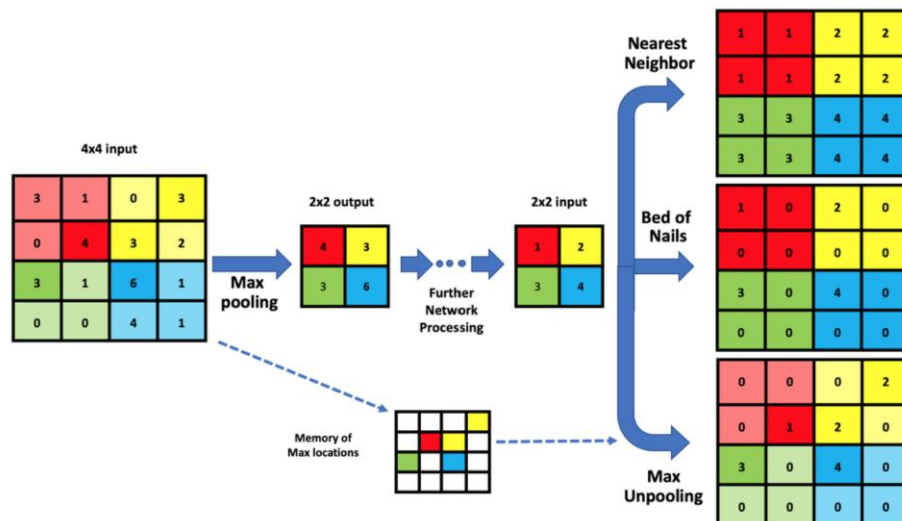
- We can subsample the feature map by taking every second element. Using a stride of two in a convolution achieves this effect while performing the convolution simultaneously.
- Max pooling selects the maximum value within each 2×2 region. This introduces a degree of translation invariance, since small shifts in the input often do not change the maximum values.
- Average (mean) pooling computes the average value within each 2×2 region.

In all cases, downsampling is applied independently to each channel, resulting in feature maps with reduced height and width but the same number of channels.

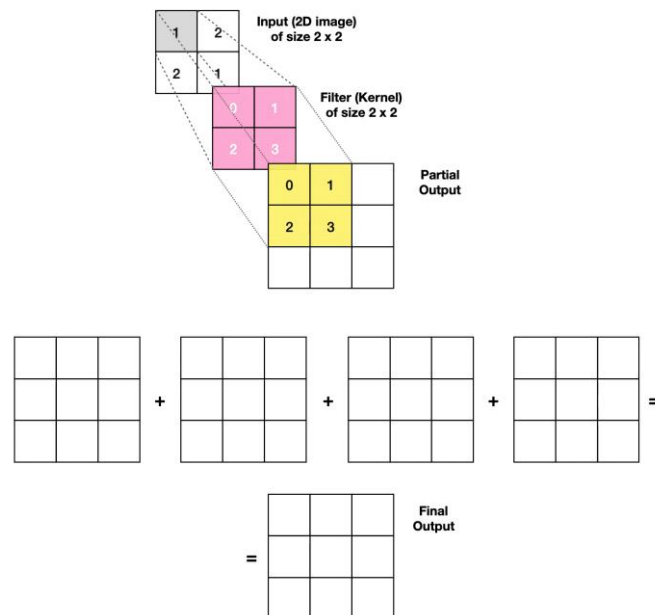


b) Upsampling

The simplest way to increase the spatial resolution of a feature map is max unpooling, which is applied when max pooling was used during downsampling. In this case, values are placed back into the positions from which they were originally selected.



Another approach is closely related to downsampling with a stride of two. In that case, the number of outputs is reduced by half, and with a kernel size of three, each output is computed as a weighted sum of three neighboring inputs. In transposed convolution, this process is effectively reversed. The number of outputs is doubled, and each input contributes to multiple output positions. When examining the corresponding weight matrix of this upsampling operation, we observe that it is the transpose of the matrix used in the corresponding downsampling convolution.



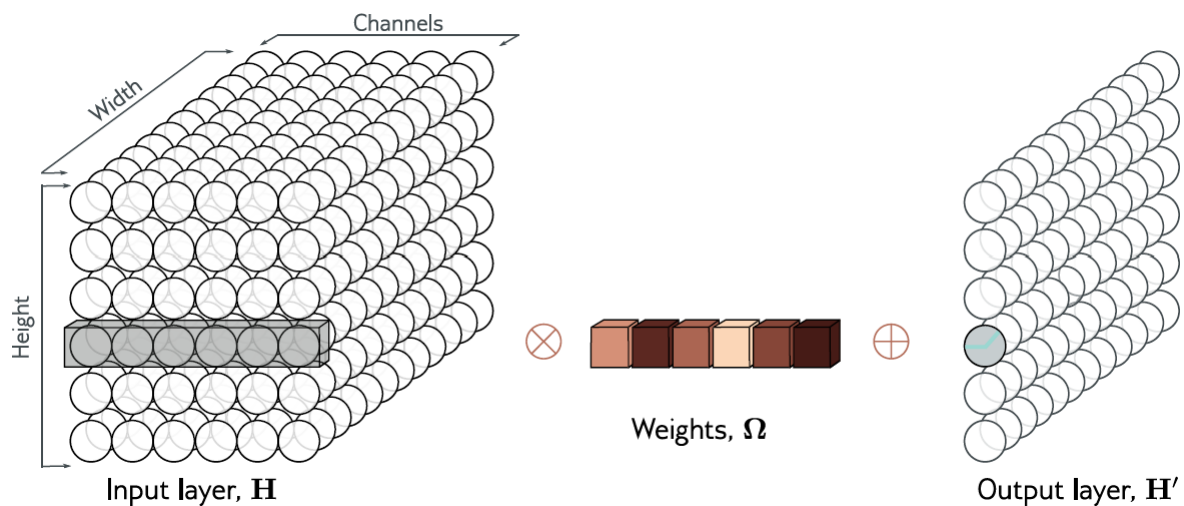
c) Changing the number of channels

Sometimes we need to change the number of channels between consecutive hidden layers without altering the spatial resolution. This is often done to merge features with another parallel branch of the network.

To achieve this, we use a 1×1 convolution. At each spatial location, the output is computed as a weighted combination of all input channels at that same position. By applying multiple such kernels with different weights, we can produce any desired number of output channels.

The corresponding convolutional weights have shape $1 \times 1 \times C_i \times C_o$, which defines a 1×1 convolution. When combined with a bias term and an activation function, this operation is equivalent

to applying the same fully connected layer independently at every spatial location.



5.3. Applications

We conclude by presenting three key computer vision applications. First, we discuss image classification using convolutional networks, where the task is to assign an image to one of a fixed set of categories. Next, we consider object detection, which involves identifying multiple objects within an image and predicting a bounding box for each one. Finally, we introduce semantic segmentation, an early approach in which each pixel is assigned a label corresponding to the object or region it belongs to.

5.3.1. Image classification

Much of the early progress in deep learning for computer vision was driven by image classification on the ImageNet dataset. The dataset contains 1.28 million training images, 50,000 validation images, and 100,000 test images, with each image labeled into one of 1,000 categories.

In most approaches, images are resized to a fixed resolution. A typical model takes a 224×224 RGB image as input and outputs a probability distribution over the 1,000 classes. The problem is highly challenging due to the large number of categories and their significant visual variability.

In 2011, before the widespread use of deep neural networks, the best systems achieved around a 25% error rate even when considering whether the correct label appeared in the top five predictions. Only five years later, deep learning models surpassed human-level performance on this benchmark.



In 2012, AlexNet became the first CNN to achieve strong performance on this task. The dataset was expanded by a factor of 2048 using (i) spatial transformations and (ii) intensity-based variations of the

input images. During testing, the network evaluated five cropped and mirrored versions of each image, and the resulting predictions were averaged.

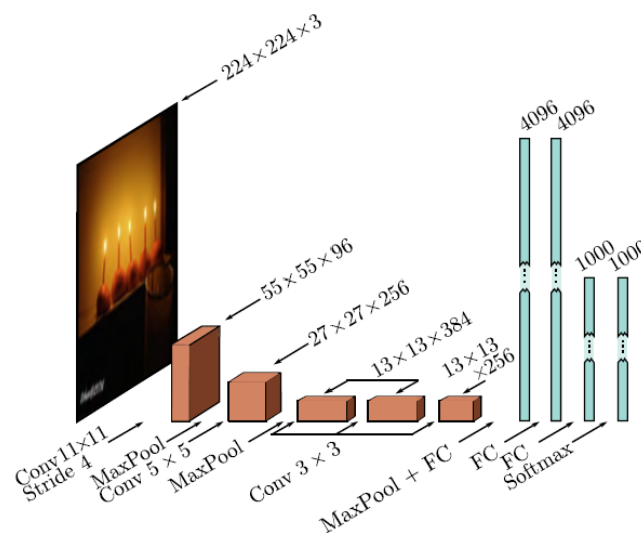
It contains eight hidden layers with ReLU activations: the first five are convolutional layers, followed by three fully connected layers.

The network begins by applying an 11×11 convolution with stride 4, producing 96 feature maps. This is followed by a max-pooling layer for downsampling. Next, a 5×5 convolution produces 256 channels, followed by another pooling step. Three additional 3×3 convolutional layers are then applied, resulting in a 13×13 representation with 256 channels.

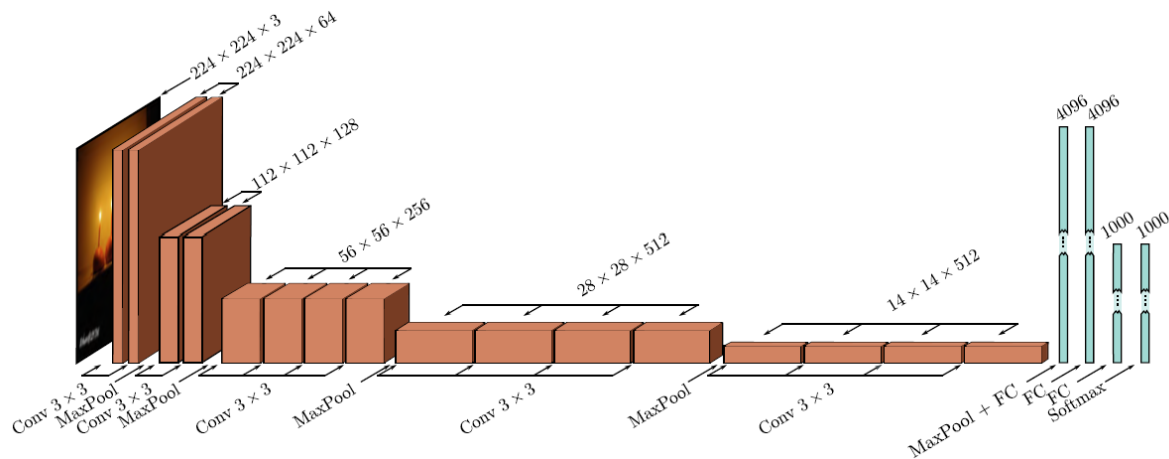
A final max-pooling operation reduces this to a $6 \times 6 \times 256$ tensor, which is flattened into a vector of length 9,216. This vector is passed through three fully connected layers with 4096, 4096, and 1000 units, respectively. The final layer uses a softmax function to produce a probability distribution over the 1,000 classes. Overall, the network has approximately 60 million parameters, the majority of which are concentrated in the fully connected layers.

Training was performed using SGD with a momentum of 0.9 and a batch size of 128. Regularization included dropout in the fully connected layers as well as L2 weight decay.

The model achieved a top-5 error rate of 16.4% and a top-1 error rate of 38.1%. At the time, this represented a major breakthrough on a problem that was previously considered extremely difficult for existing methods. This result demonstrated the power of deep learning and marked the beginning of the modern AI era.



The VGG network was also designed for image classification on the ImageNet task and achieved significantly improved performance, with a 6.8% top-5 error rate and a 23.7% top-1 error rate. Like AlexNet, it is built from a sequence of convolutional and max-pooling layers, where the spatial resolution gradually decreases while the number of channels increases. These layers are followed by three fully connected layers. VGG was trained using data augmentation, weight decay, and dropout. While there were some minor differences in training, the main distinction between AlexNet and VGG is the depth of the network. VGG is much deeper and contains about 144 million parameters.



For several years, improvements in performance on this task were largely driven by increasing network depth, highlighting the importance of depth in neural networks.

Model	Year	Key Contribution
Inception	2014	Introduced the Inception module with multi-scale convolutions and dimension reduction using 1×1 convolutions.
ResNet	2015	Introduced residual connections to enable very deep networks (up to 152 layers).
MobileNet	2017	Introduced depth-wise separable convolutions for efficient mobile networks.
EfficientNet	2019	Proposed compound scaling to balance depth, width, and resolution for efficient model scaling.

1) Inception network

Despite being introduced in 2014, the Inception models remain among the most effective neural networks for image classification and detection. Their foundational paper, *Going Deeper with Convolutions*, is considered a classic in machine learning literature.

The motivation behind the Inception architecture was to improve the representational power of deep neural networks while keeping computational costs manageable. Traditional CNNs require manually choosing the right filter size for each layer, which can limit the ability to capture multi-scale features. Consider the images of peacocks below. The area occupied by the peacock varies in each image, making it challenging to choose the appropriate kernel size. A larger kernel size is used to capture the global distribution of the image, while a smaller kernel size focuses on capturing more localized details.



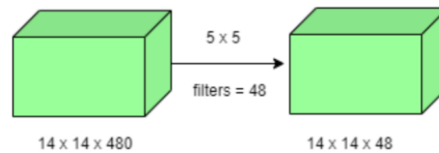
Core Elements of the Inception Architecture

a. 1x1 convolution

Before diving into the Inception model, it's important to understand a key concept used within it: the 1x1 convolution. A 1x1 convolution maps each input pixel and its associated channels to a corresponding output pixel. It serves as a dimensionality reduction tool, helping to reduce computation by simplifying the input data.

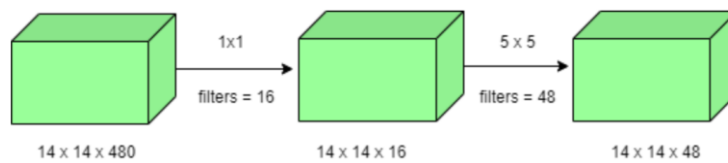
Example:

For example, let's consider performing a 5x5 convolution without utilizing a 1x1 convolution, as shown below:



Number of operations involved here is $(5 \times 5 \times 480) \times (14 \times 14 \times 48) = 112.9M$

Using 1x1 convolution:



Number of operations for 1x1 convolution = $(1 \times 1 \times 480) \times (14 \times 14 \times 16) = 1.5M$

Number of operations for 5x5 convolution = $(5 \times 5 \times 16) \times (14 \times 14 \times 48) = 3.8M$

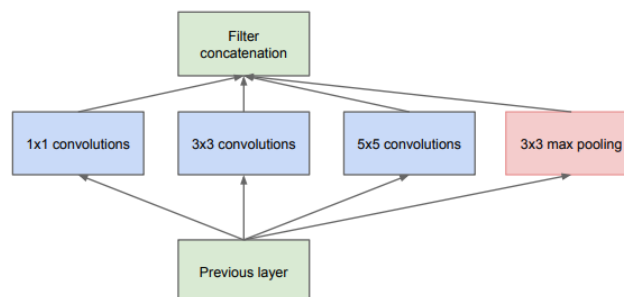
After addition we get, $1.5M + 3.8M = 5.3M$, which is significantly smaller than 112.9M. Therefore, 1x1 convolutions can help reduce the model size, which can also contribute to mitigating the overfitting issue.

The Inception network architecture allows for the use of filters of various sizes without increasing the network's depth. Instead of connecting them sequentially, the different filters are applied in parallel.

b. Inception block

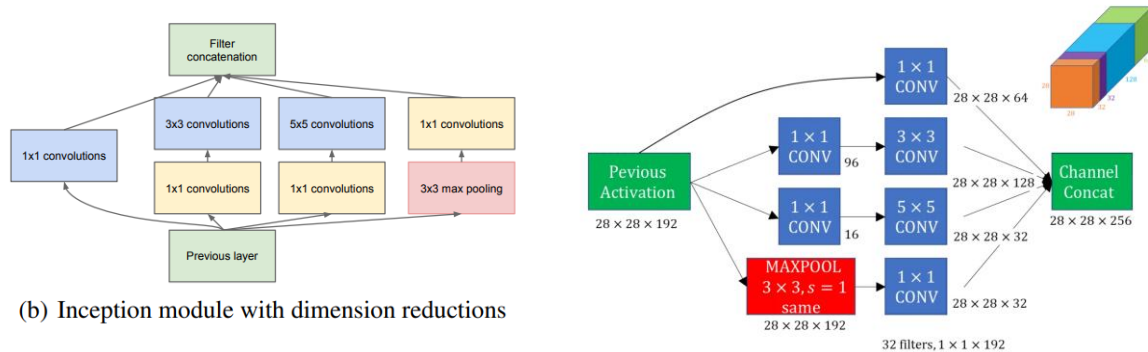
A naive Inception block is an early version of the Inception architecture designed to capture multi-scale features from the input by applying several convolution operations in parallel. Instead of choosing a fixed filter size, the block simultaneously applies multiple types of convolutions, typically 1x1, 3x3, and 5x5, along with a 3x3 max pooling operation, all on the same input.

Each of these parallel outputs captures different kinds of information (fine details with 1x1, medium patterns with 3x3, and larger patterns with 5x5). The outputs are then concatenated along the depth (channel) dimension, allowing the next layer to access multi-scale features.



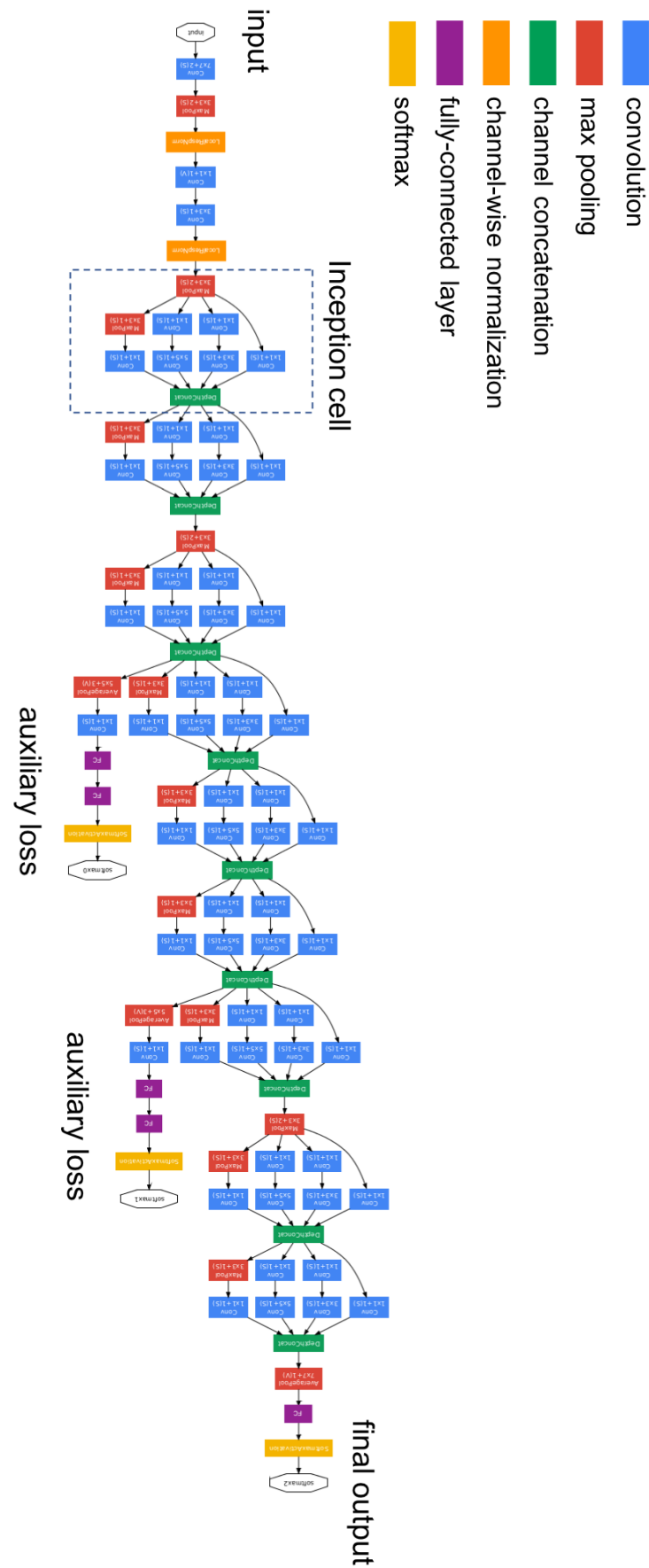
(a) Inception module, naïve version

However, this structure can become computationally expensive very quickly. To address this issue, 1×1 convolutional layers are introduced before the larger convolutional layers (such as 3×3 , 5×5 , etc.). These 1×1 layers reduce the number of channels, thereby lowering the computational cost.



c. The architecture of Inception V1

The GoogLeNet architecture consists of 22 layers in total (27 including the pooling layers). It is built using the dimension-reduced Inception module, which gives rise to the neural network architecture known as Inception v1, or GoogLeNet. This model includes 9 Inception modules arranged sequentially. At the end of the network, fully connected layers are replaced with global average pooling, which computes the average of each feature map. This modification significantly reduces the total number of parameters. As a result, Inception Net outperforms previous CNN models, achieving top-5 accuracy on ImageNet while dramatically lowering computational costs without sacrificing speed or accuracy.



d. What distinguishes the Inception V3 network from the Inception V1 network?

Inception V3 is an enhanced version of the V1 module, incorporating techniques such as factorizing larger convolutions into smaller ones (for example, breaking a 5x5 convolution into two 3x3 convolutions) and using asymmetric factorizations (such as splitting a 3x3 filter into a 1x3 and a 3x1 filter). These factorizations are applied to reduce the number of parameters used in each Inception module.

First factorization:

For a 5x5 convolution: $Params_{5 \times 5} = 5 \times 5 \times C_{in} \times C_{out} = 25 \times C_{in} \times C_{out}$

For two 3x3 convolutions: $Params_{2 \times (3 \times 3)} = 3 \times 3 \times C_{in} \times C_{out} + 3 \times 3 \times C_{out} \times C_{out} = 9 \times C_{in} \times C_{out} + 9 \times C_{out}^2$

$$R = \frac{Params_{2 \times (3 \times 3)}}{Params_{5 \times 5}} = \frac{9 \times C_{in} \times C_{out} + 9 \times C_{out}^2}{25 \times C_{in} \times C_{out}}$$

If $C_{in} = C_{out}$:

$$R = \frac{18 \times C_{out}^2}{25 \times C_{out}^2} = 0.72$$

Two 3x3 convolutions use only 72% of the parameters compared to a direct 5x5 convolution. In other words, we save about 28% of the parameters for the same receptive field.

Second factorization:

For a 3x3 convolution: $Params_{3 \times 3} = 3 \times 3 \times C_{in} \times C_{out} = 9 \times C_{in} \times C_{out}$

For parallel 3x1 and 1x3 convolutions:

$$Params_{3 \times 1} = 3 \times 1 \times C_{in} \times C_{out} = 3 \times C_{in} \times C_{out}$$

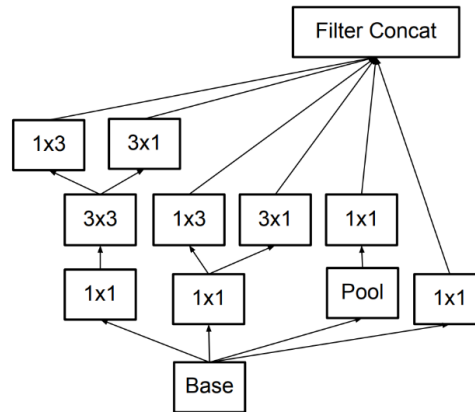
$$Params_{1 \times 3} = 1 \times 3 \times C_{in} \times C_{out} = 3 \times C_{in} \times C_{out}$$

$$Params_{parallel} = 3 \times C_{in} \times C_{out} + 3 \times C_{in} \times C_{out} = 6 \times C_{in} \times C_{out}$$

$$R = \frac{Params_{parallel}}{Params_{3 \times 3}} = \frac{6 \times C_{in} \times C_{out}}{9 \times C_{in} \times C_{out}} = \frac{6}{9} \approx 0.666$$

We save about 33% of the parameters

Below is an image of the Inception V3 module.



2) Residual Network

In recent years, the field of Computer Vision has witnessed significant breakthroughs, particularly with the emergence of deep Convolutional Neural Networks (CNNs). These models have achieved state-of-the-art performance on tasks such as image classification and recognition. To tackle increasingly complex challenges and further boost accuracy, researchers have progressively built deeper neural networks by adding more layers. However, this approach has revealed a critical limitation: as networks

become deeper, they become harder to train, and their performance may plateau or even deteriorate. This is where ResNet (Residual Networks) comes into play, offering a solution to these issues.

What is ResNet?

ResNet, short for Residual Network, is a specialized type of neural network introduced in 2015 by Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun in their landmark paper "*Deep Residual Learning for Image Recognition*." The ResNet models gained remarkable success.

Vanishing Gradient Problem: As the depth of a neural network increases, the gradients of the loss function with respect to the weights can become extremely small during backpropagation, a phenomenon known as the vanishing gradient problem. This hinders the learning process in the deeper layers, as the weight updates become too small to drive meaningful learning.

Performance Plateau: As the number of layers in a neural network increases, the training error often saturates and ceases to improve. This indicates that adding more layers does not contribute meaningfully to reducing the training error.

Accuracy Degradation: Surprisingly, adding more layers can lead to an increase in validation error, resulting in poorer performance on unseen data. This suggests that deeper networks may overfit or become harder to optimize effectively.

Core Elements of the ResNet Architecture

a. Residual Block:

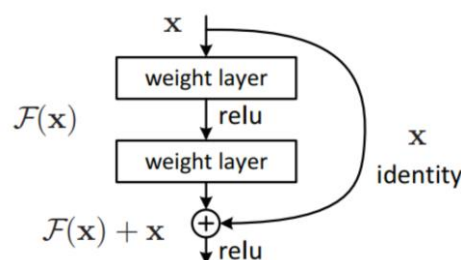
Residual blocks are the fundamental building blocks of a Residual Neural Network (ResNet). In a traditional neural network, the input is sequentially transformed through a series of convolutional layers followed by activation functions. In contrast, a ResNet introduces a shortcut, or *residual connection*, by adding the original input of the block directly to its output. The output of a residual block, denoted as y , can be expressed as:

$$y = H(x) = F(x) + x$$

The network combines this residual $F(x)$ with the input x using a shortcut.

A challenge arises with this approach when the dimensions of the input differ from those of the output, a situation that can occur due to convolutional or pooling operations. When $f(x)$ and x have mismatched dimensions, there are two common strategies to address this:

- One way to handle the mismatch in dimensions is by padding the skip connection with additional zero entries to match the required size.
- Alternatively, the *projection shortcut* method can be used, where a 1×1 convolution is applied to the input to align its dimensions with the output.

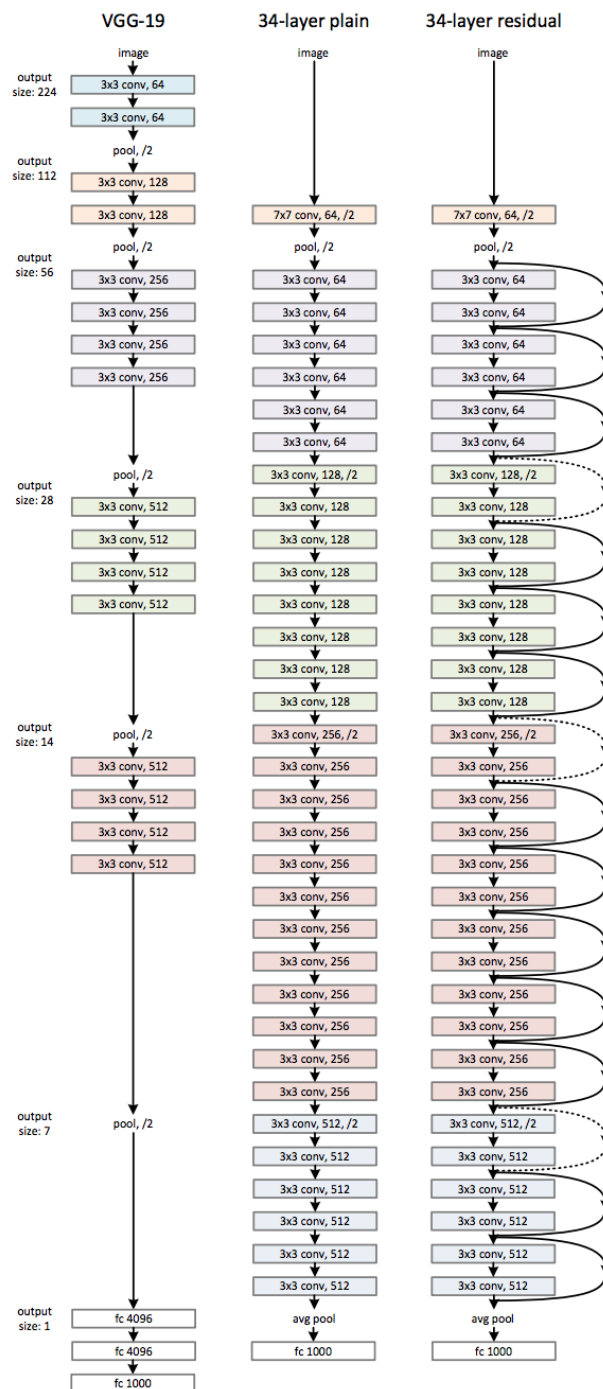


- The skip connections in ResNet address the vanishing gradient problem in deep neural networks by providing an alternate shortcut path for gradients to flow through.
- Imagine we have both a shallow and a deep network, each mapping an input x to an output y through a function $H(x)$. Our goal is for the deep network to perform at least as well as the shallow one,

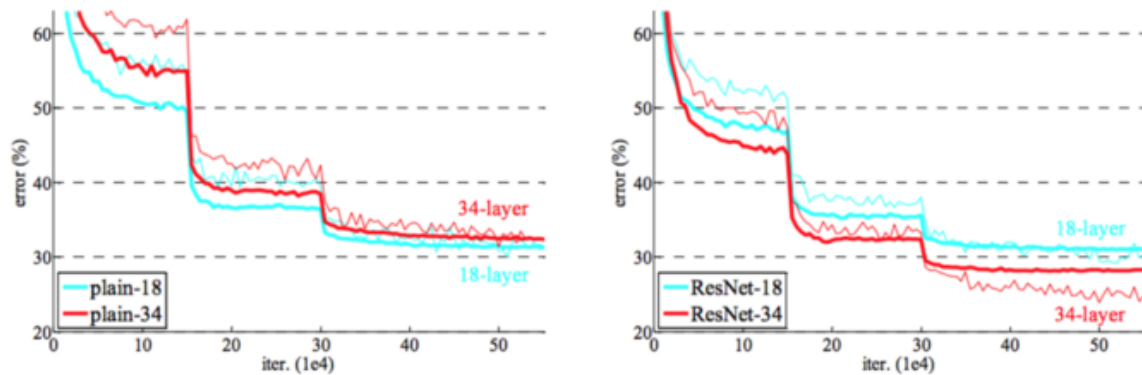
without the performance degradation often observed in plain networks (those without residual connections). One way to achieve this is by enabling the additional layers in the deep network to learn an identity mapping, effectively passing their inputs directly to the outputs. This ensures that even with extra layers, the performance does not degrade.

ResNet architecture

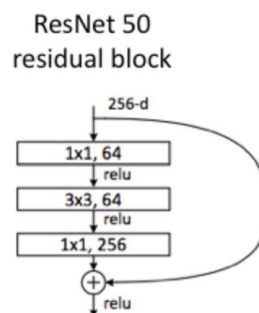
ResNet builds upon a 34-layer plain network architecture, inspired by VGG-19, by introducing shortcut connections. These connections transform the plain architecture into a residual network, as illustrated in the figure below.



Each colored block represents a sequence of convolutions operating at the same spatial resolution. Feature maps are periodically down-sampled using strided convolutions, while the channel depth is increased to maintain a constant computational cost per layer. Dotted lines indicate residual connections where a 1×1 convolution is used to project the input to match the dimensions of the subsequent block. ResNet has substantially improved the performance of deep neural networks, especially as the number of layers increases. The plot below shows the error percentage in comparison to networks with plain (non-residual) layers.



In the ResNet-50 model, each two-layer residual block is replaced by a three-layer bottleneck block, where 1×1 convolutions are used to first reduce and then restore the channel depth. This design lowers the computational cost of the intermediate 3×3 convolution.



3) MobileNet

MobileNet is a lightweight and efficient CNN designed for mobile vision applications, offering strong performance with low computational cost. MobileNet is a family of neural networks optimized for efficient inference on mobile and embedded devices. The original MobileNet V1 introduced depth-wise separable convolutions, a technique that significantly reduces the computational cost compared to traditional convolutions.

Core Elements of the ResNet Architecture

a. Depth-wise separable convolution

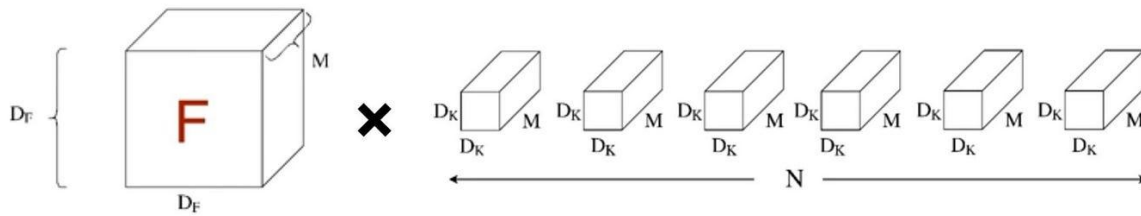
Depth-wise separable convolution is based on the idea that a filter's spatial and depth dimensions can be decoupled, hence the term "separable."

A depth-wise separable convolution is made from two operations:

1. Depth-wise convolution.
2. Point-wise convolution.

Depth-wise convolution: Depth-wise convolutions apply a single filter to each input channel individually, unlike standard convolutions where each filter operates across all input channels.

A standard convolution is defined as follows:

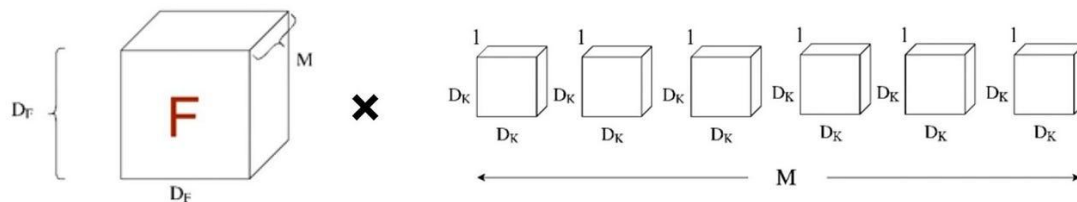


- D_K is the spatial dimension of the kernel
- D_F is the spatial width and height of the input feature map
- M is the number of input channels
- N is the number of output channels

The computational cost can be calculated as: $(D_K \times D_K \times M) \times (D_F \times D_F \times N)$

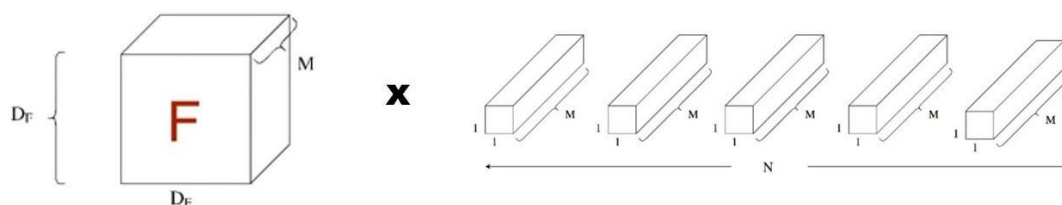
In standard convolution, the computational cost grows proportionally with the number of input and output channels, as well as the spatial dimensions of both the input feature map and the convolution kernel.

In the case of depth-wise convolution, as illustrated in the image below, the input feature map has dimensions $D_F \times D_F$, and there are M kernels, each with a channel size of 1.



As shown in the image above, the total computational cost can be clearly computed as: $D_K \times D_K \times M \times D_F \times D_F$

Point-wise Convolution: Since depth-wise convolution only filters each input channel independently without combining them to create new features, an additional layer called the point-wise convolution is introduced. It applies a 1×1 convolution to compute linear combinations of the depth-wise convolution outputs.



As shown in the image, let's now calculate the computational cost again: $M \times D_F \times D_F \times N$

Therefore, the total computational cost of depth-wise separable convolutions can be computed as:

$$D_K \times D_K \times M \times D_F \times D_F + M \times D_F \times D_F \times N$$

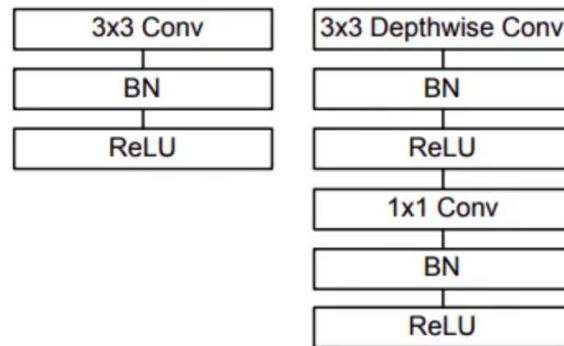
When compared to the computational cost of standard convolution, we observe a reduction in computation, which can be expressed as:

$$\frac{D_K \times D_K \times M \times D_F \times D_F + M \times N \times D_F \times D_F}{D_K \times D_K \times M \times N \times D_F \times D_F} = \frac{1}{N} + \frac{1}{D_K^2}$$

To put this into perspective and evaluate the effectiveness of depth-wise separable convolution, let's consider an example. Let's set $N=1024$ and $D_K=3$, and substitute these values into the equation. This results in 0.112, meaning that standard convolution requires 9 times more multiplications than depth-wise separable convolution.

b. The architecture of MobileNet V1

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5× Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
	Conv / s1	$1 \times 1 \times 512 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$



From the figure above, we can observe that each convolutional layer is followed by batch normalization and a ReLU activation. Additionally, an average pooling layer is introduced just before the fully connected layer to reduce the spatial dimensions to 1. It's also worth noting that this architecture consists of 28 layers, counting both depth-wise and point-wise convolutions as separate layers.

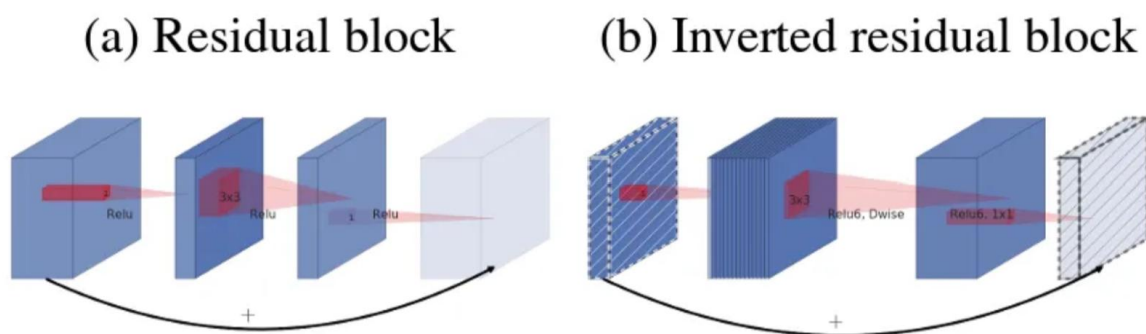
c. Inverted Residuals (MobileNet V2)

In ResNet, a residual block reduces the feature dimension and then expands it again. Unlike ResNet, in MobileNetV2, the opposite procedure is applied, first expansion, then reduction. That's why it's called inverted residual. Suppose the input from the depth-wise separable convolution has dimensions $8 \times 8 \times 64$.

Step 1: Expansion (Increasing the Number of Channels): a 1×1 point-wise convolution is applied to expand the number of channels from 64 to 256. Multiplications for expansion: $64 \times 256 \times 8 \times 8 = 1,048,576$.

Step 2: Depth-wise Convolution (Applying a Separate Filter to Each Channel): We apply a depth-wise convolution to the $8 \times 8 \times 256$ bottlenecked output, where a 5×5 filter is applied independently to each channel. The number of multiplications for the depth-wise convolution is $5 \times 5 \times 64 \times 8 \times 8 = 102,400$.

Step 1: Projection/Bottleneck (Reducing Dimensionality): To reduce computation, a 1×1 point-wise convolution is applied to decrease the number of channels (for example, from 256 to 64), effectively compressing the information into a smaller bottleneck. As a result, the output after this step has dimensions $8 \times 8 \times 64$. Multiplications for projection: $256 \times 64 \times 8 \times 8 = 131,072$

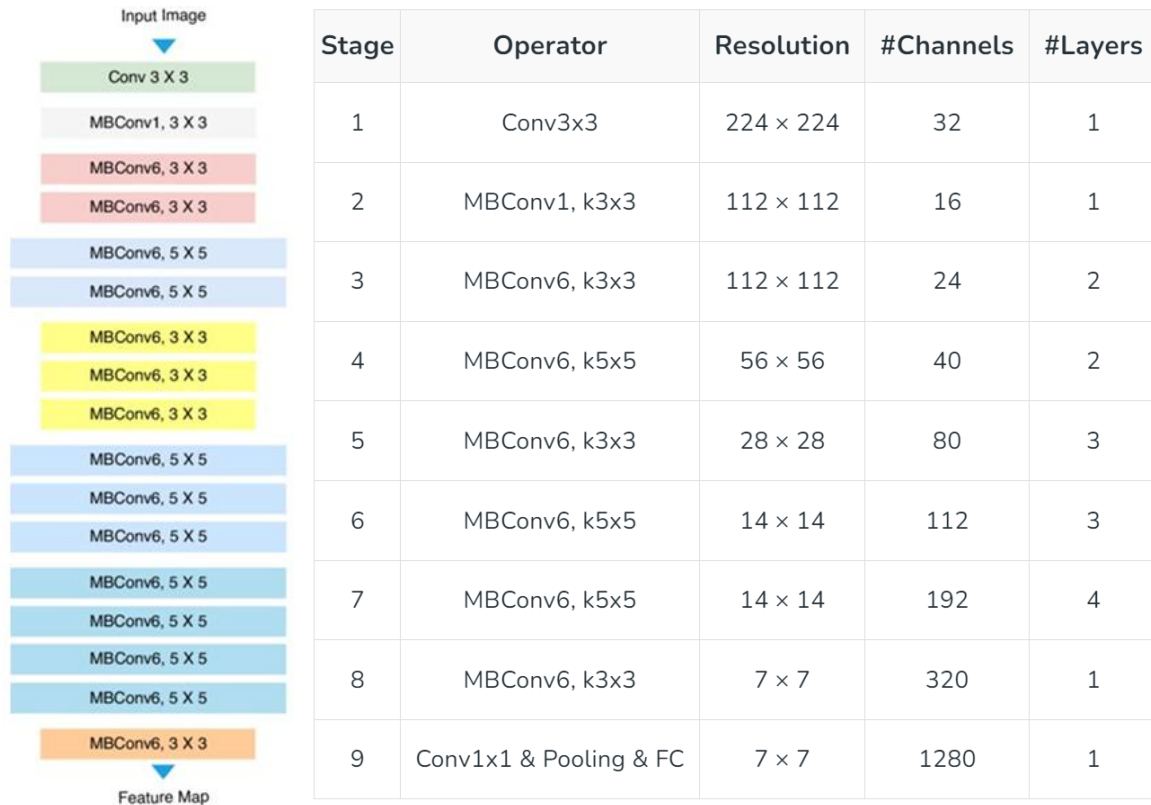


4) EfficientNet

When convolutional neural networks were initially developed, they were designed with a fixed resource budget. As more computational resources become available, these networks were often scaled up to improve accuracy. For example, a ResNet-18 can be scaled to a ResNet-200 by adding more layers. While this traditional scaling approach typically improves performance on benchmark datasets, it is largely ad-hoc: some models are scaled by increasing depth, others by increasing width, and some by using higher-resolution input images. This manual, trial-and-error process demands significant effort and often yields only marginal gains. To address this, the authors of EfficientNet proposed a more principled and systematic method for scaling CNNs to achieve better accuracy and efficiency.

a. Overview of the EfficientNet-B0 Architecture

- 1- **Stem:** The network begins with a standard convolutional layer using 32 filters of size 3×3 and a stride of 2, followed by batch normalization and a ReLU6 activation.
- 2- **Body:** Composed of a sequence of MBConv blocks, each with varying configurations.
- 3- **Head:** The head consists of a final convolutional block, followed by global average pooling and a fully connected layer with a softmax activation for classification.



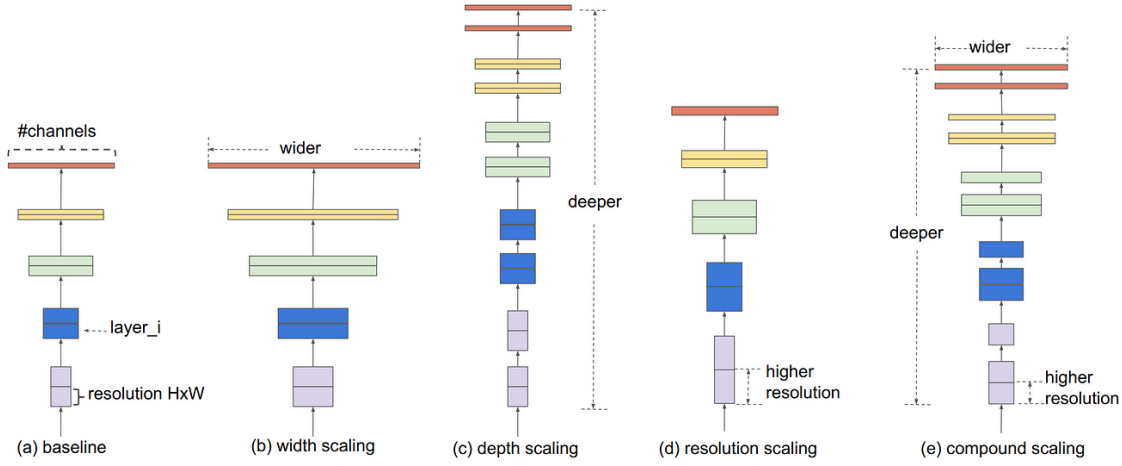
EfficientNet introduces a technique called compound scaling to scale models in a simple yet effective way. Rather than scaling width, depth, or resolution independently and arbitrarily, compound scaling uniformly adjusts all three dimensions using a fixed set of scaling coefficients. By applying this method, along with AutoML, the authors developed seven models of varying sizes, each achieving state-of-the-art accuracy while being significantly more efficient than previous convolutional networks.

Compound Scaling Method

The authors carefully analyzed how each individual scaling strategy: width, depth, and resolution impacts the model's performance and efficiency. Their findings showed that while scaling any single dimension can improve performance to some extent, the best overall gains come from scaling all three dimensions together in a balanced way, while also considering available computational resources. This insight led to the development of the compound scaling method.

The images below illustrate the different scaling strategies:

- **Baseline:** The original network without any scaling.
- **Width Scaling:** Increasing the number of channels in each layer.
- **Depth Scaling:** Increasing the number of layers in the network.
- **Resolution Scaling:** Increasing the input image size.
- **Compound Scaling:** Simultaneously scaling width, depth, and resolution based on a carefully designed formula.



Depth (d): Scaling the depth of a network is one of the most widely used strategies. The concept is straightforward: deeper convolutional networks are capable of capturing richer, more complex features and often generalize better. However, this approach introduces a challenge, the vanishing gradient problem.

Width (w): Increasing a network's width is commonly used in smaller models. A wider model can capture more fine-grained features, but overly wide networks often struggle to learn higher-level abstractions.

Image Resolution (r): Using higher-resolution images allows the model to capture finer and more detailed patterns. While earlier models typically used 224×224 images, more recent models have shifted toward higher resolutions. However, increasing image resolution also significantly raises the computational cost.

The **compound scaling method** uniformly scales a network's depth, width, and resolution in a balanced way using a predefined compound coefficient ϕ . The mathematical expression for compound scaling is as follows:

$$\begin{aligned}
 \text{depth: } d &= \alpha^\phi \\
 \text{width: } w &= \beta^\phi \\
 \text{resolution: } r &= \gamma^\phi \\
 \text{s.t. } \alpha \cdot \beta^2 \cdot \gamma^2 &\approx 2 \\
 \alpha \geq 1, \beta \geq 1, \gamma &\geq 1
 \end{aligned}$$

- α : Scaling factor for network depth (usually between 1 and 2)
- β : Scaling factor for network width (typically between 1 and 2)
- γ : Scaling factor for input resolution (generally between 1 and 1.5)
- ϕ : The compound coefficient (a positive integer) that controls the overall degree of scaling.

This approach determines how much to scale the depth, width, and resolution to achieve optimal performance.

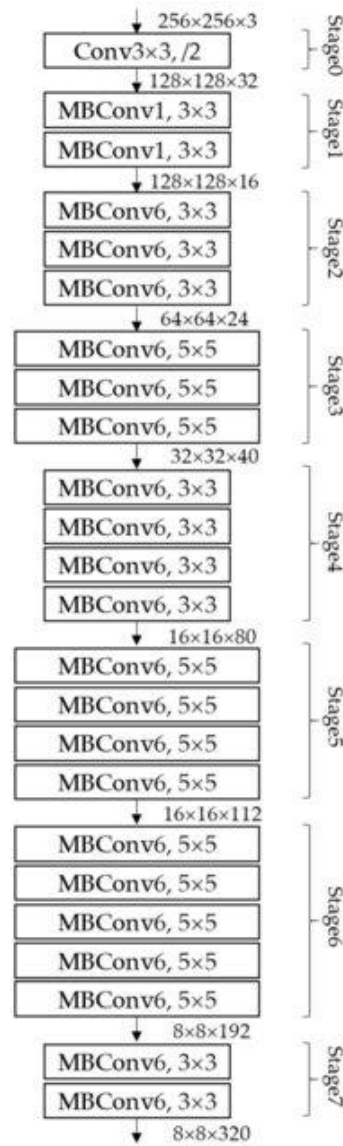
Example:

$\alpha=1.2 \rightarrow$ depth increases by 20% for every ϕ

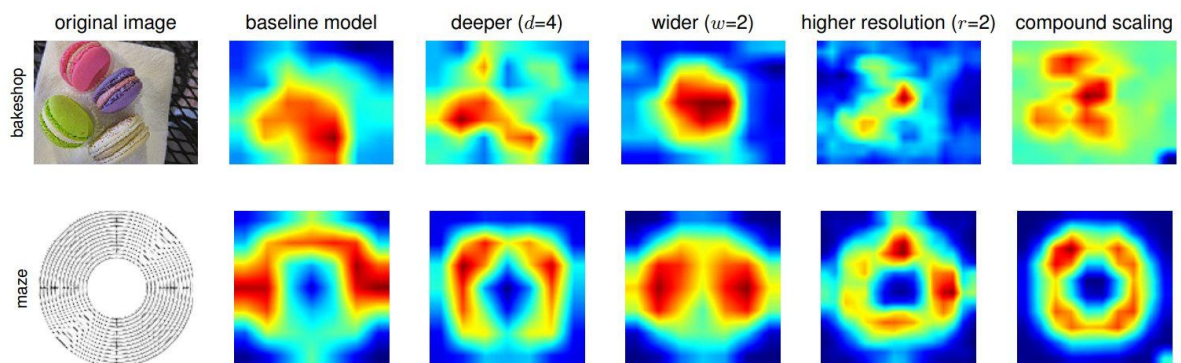
$\beta=1.1 \rightarrow$ width increases by 10% for every ϕ

$\gamma=1.15 \rightarrow$ resolution increases by 15% for every ϕ

For $\phi = 1$, we obtain EfficientNet-B1:



Moreover, we highlight the benefit of compound scaling through Class Activation Map (MAP) visualizations.



Object Detection

Object detection is a computer vision task that identifies and locates objects within an image or video. It involves drawing bounding boxes around detected objects, enabling us to determine their position and track their movement within a scene.



Input Image



Sample Output (Source Ref. 7)

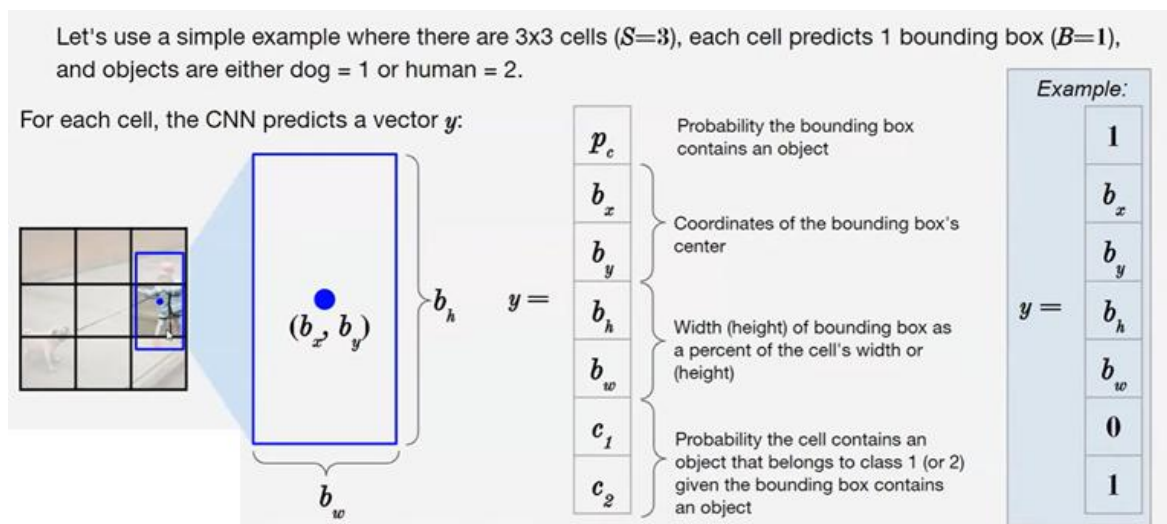
YOLO

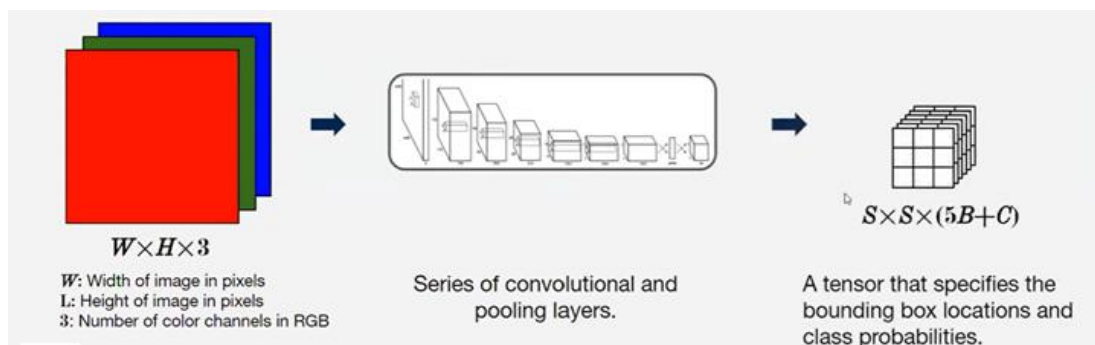
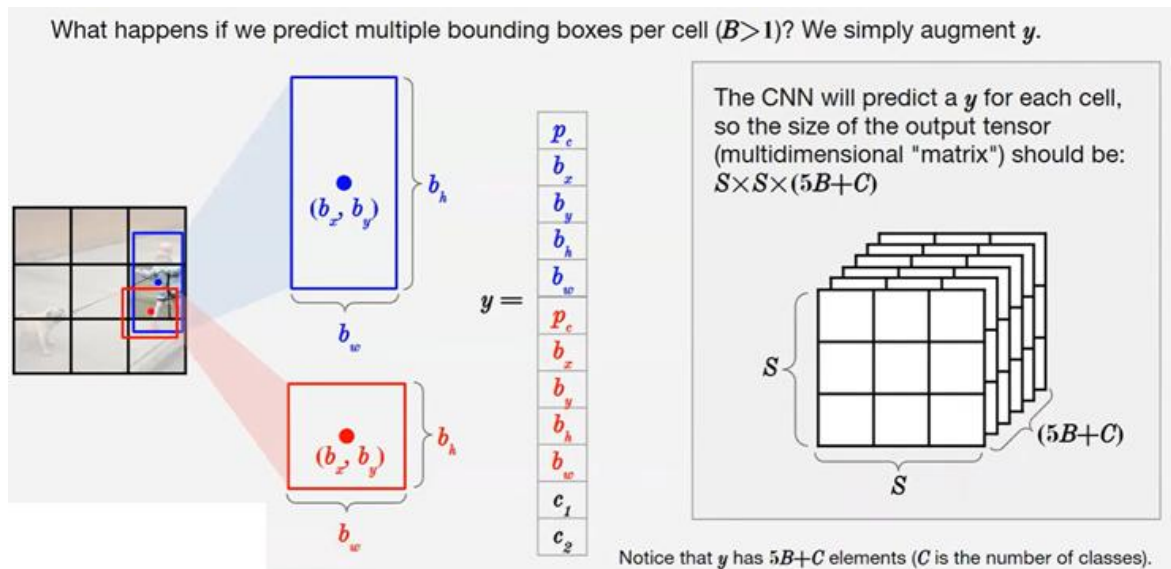
YOLO is a single-stage object detection algorithm that consists of 24 convolutional layers followed by two fully connected layers. The convolutional network simultaneously predicts both bounding box coordinates and class probabilities, answering both the "what" and the "where" in one step. This end-to-end, single-iteration approach leads to faster detection and often higher accuracy, as bounding boxes and class labels are predicted together.

YOLO steps

In a single pass over the image, YOLO follows three key steps to detect and classify objects:

1. **Grid cells:** The image or frame is divided into a grid of fixed size, typically $S \times S$. Each grid cell is responsible for detecting objects whose centers fall inside it. The illustration above shows how these grid-based bounding boxes are generated.
2. **Bounding Box Regression:** A bounding box is drawn around an object to outline its location within the image. Each box is defined by attributes: objectness, height b_h , width b_w , class label (the object's category), and the coordinates of its center (b_x, b_y).





3. **Intersection over union:** It describes how boxes overlap. The IoU equals 1 if the predicted bounding box is the same as the real box. This mechanism eliminates bounding boxes that are not equal to the real box. IoU is calculated as the ratio of 'area of overlap' to 'area of the union'. Finally, by Non-Maximal Suppression, YOLO suppresses all bounding boxes with lower probability scores to achieve final bounding boxes.

$$\text{IoU} = \frac{\text{area of overlap}}{\text{area of union}} = \frac{\text{area of overlap}}{\text{area of union}}$$

During Training: An object is assigned to a grid cell if the center of the object falls inside that cell. We pick the bounding box with the highest IoU (Intersection over Union) with the ground-truth box.

During Inference: Non-maximum suppression (NMS) is applied. Each bounding box has a confidence score (which represents how likely the box contains an object) and a class score. The final score for each box is calculated by:

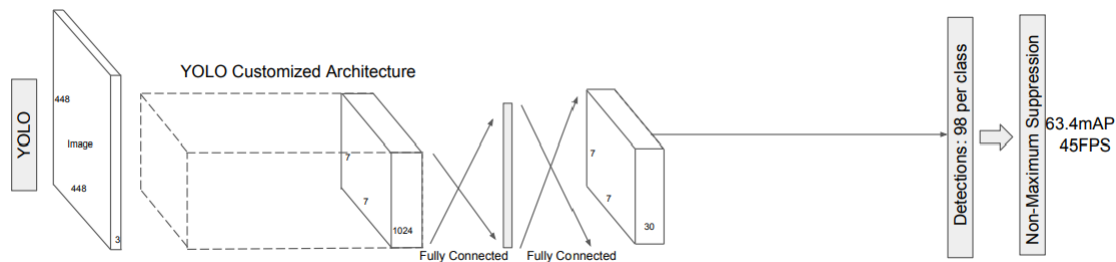
$$\text{Class-specific score} = \text{Confidence} \times \text{Class Probability}$$

This score determines how confident the model is about the presence of an object in that box.

The boxes are sorted in descending order based on the highest class-specific score. Each class is handled independently during NMS, so boxes belonging to different classes are not compared in terms of overlap. For each class, the highest-scoring box is selected and for each of the remaining boxes that belong to the same class, calculate the IoU with the selected box. If the IoU is above a given threshold (say 0.5),

those boxes are discarded. Repeat for the next highest-scoring box.

YOLO Architecture



The YOLO model consists of three main components: the backbone, neck, and head. The backbone is a series of convolutional layers that extract key features from the image. It is initially trained on a classification dataset like ImageNet and typically at a lower resolution than the detection model, as object detection requires more detailed information than classification. The neck uses the features from the backbone's convolutional layers and processes them with fully connected layers to predict class probabilities and bounding box coordinates. The head is the final output layer, producing an $S \times S \times (C+B \times 5)$ tensor, where S is the grid size, C is the number of classes, and B is the number of bounding boxes. For example, in the original YOLO paper, the output is a $7 \times 7 \times 30$ tensor, where $S=7$, $C=20$, and $B=2$. Together, these components work to extract features, classify objects, and predict bounding boxes.

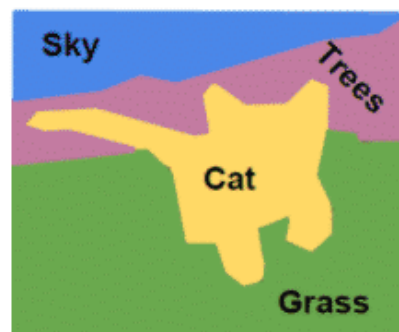
Semantic segmentation

Semantic segmentation, also referred to as dense prediction, aims to assign a class label to each pixel in the input image, representing a specific object or region. This task is particularly useful when spatial relationships and interactions within the scene are important, such as in autonomous driving.

It is important to note that semantic segmentation does not distinguish between individual instances of the same class. For example, even if two cars differ in make and color, both would be labeled as 'Car' and treated as a single category.



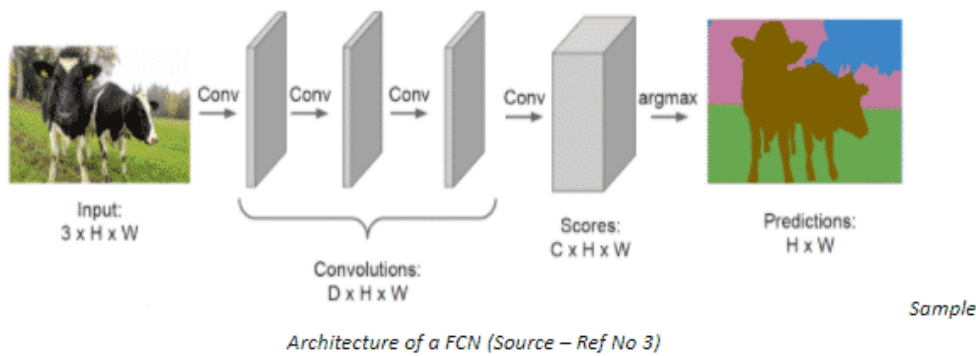
Input Image



Expected Output (Source – Ref. 3)

One approach to solving the problem of semantic segmentation is to stack multiple convolutional layers with consistent padding to preserve the dimensions, ultimately producing a final segmentation map. In this way, the model learns the mapping from the input image to its corresponding segmentation map by

successively transforming the feature maps.



While this architecture seems promising, there is a major issue. Using the same padding across all convolutional layers is intended to keep the output image dimensions the same as the input. However, this approach, which preserves full resolution, becomes computationally expensive. One option is to reduce the number of layers, but this would significantly hurt the model's performance.

In classification tasks, we don't face this dilemma because we're only concerned with identifying the presence of a single object. Losing spatial information about the object's location doesn't impact the result. This allows for periodic down-sampling through pooling. However, this isn't suitable for semantic segmentation. For example, when segmenting a highway image for autonomous vehicle training, you wouldn't want important elements like vehicles to be cropped out in the final output, as precise localization is crucial.

A widely used solution to the issue in the previous architecture is the Fully Convolutional Network (FCN), which incorporates both down-sampling and up-sampling. In the first half of the model, the spatial resolution of the image is down-sampled to create complex feature maps. As each convolution progresses, finer details of the image are captured, enabling effective discrimination between different classes. However, during this process, spatial location information is lost. To recover this, the model applies an up-sampling procedure after down-sampling, which takes the lower-resolution feature maps as input and outputs a high-resolution segmentation map.

11.1.1. Fully Convolutional Network

The architecture of Fully Convolutional Networks (FCNs) is specifically designed to efficiently handle semantic segmentation tasks while preserving spatial information across the network. This section will explore the essential components and architectural details of FCNs.

FCNs are capable of processing input images of arbitrary sizes, which is a significant advantage over traditional CNNs that require fixed-size inputs. The input to an FCN is typically a multi-channel image (e.g., RGB), and the output is a segmentation map with the same spatial dimensions as the input, where each pixel is assigned a class label.

Input shape: (H, W, C)

- H: Height of the image
- W: Width of the image
- C: Number of input channels (e.g., 3 for RGB)

Output shape: (H, W, N)

- H: Height of the image
- W: Width of the image
- N: Number of classes for segmentation

11.1.1.1. FCN architecture

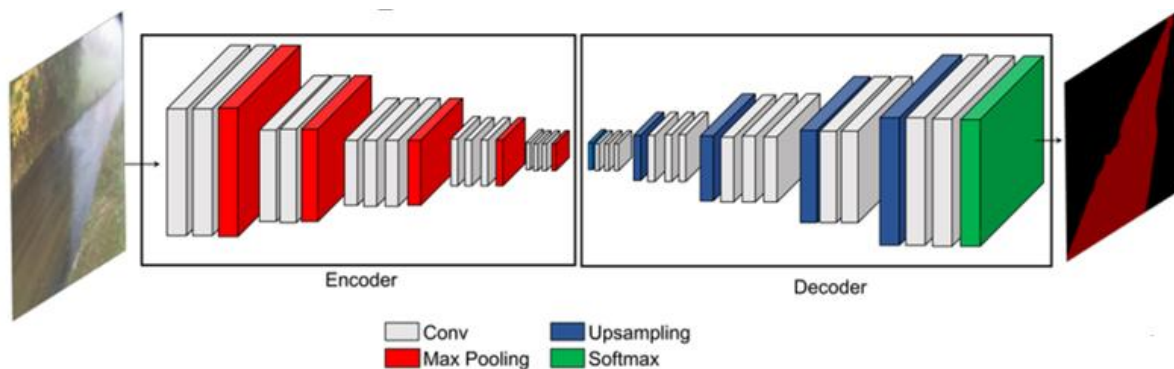
FCNs are composed of several key types of layers:

- **Convolutional Layers:** These layers form the core of the network, extracting features from the input image. Multiple convolutional layers are stacked to learn hierarchical feature representations.
- **Pooling Layers:** These layers reduce spatial dimensions and expand the receptive field. Common types include max pooling and average pooling.
- **Up-sampling Layers:** These layers restore the spatial dimensions that were reduced by pooling, typically using techniques like transposed convolutions or bilinear interpolation.

A typical FCN architecture is divided into two main components:

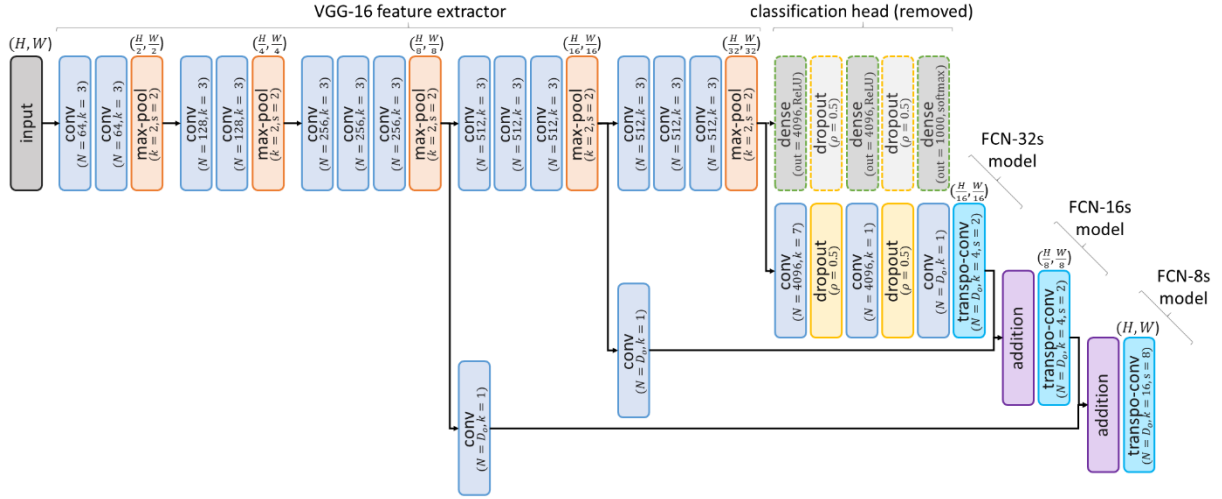
- **Encoder:** Similar to a traditional CNN, the encoder consists of convolutional and pooling layers that gradually reduce spatial dimensions while increasing the number of feature channels.
- **Decoder:** The decoder up-samples the feature maps back to the original input resolution, often using techniques like transposed convolutions or bilinear interpolation.

Below is a simplified representation of an FCN architecture:



11.1.1.2. Skip Connections

Skip connections are a vital component of FCNs, enabling the network to merge coarse, semantic information from deeper layers with fine, spatial details from shallower layers. This integration enhances the accuracy of segmentation boundaries.



11.3.2.3. Training

FCNs are trained in an end-to-end manner using backpropagation. The loss function commonly employed is pixel-wise cross-entropy:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic})$$

Where:

- N is the total number of pixels
- C is the number of classes
- y_{ic} is the true label for pixel i and class c
- $\hat{y}_{ic}$ is the predicted probability for pixel i and class c

The network parameters are updated using gradient descent or its variants to minimize this loss function.